

Atividade Estrutura de Dados - Pilhas

Parte 1 — Implementação

Exercício 1: Classe Pilha

```
public class Pilha {
    private String[] elementos;
    private int tamanho;
    private static final int CAPACIDADE = 100;

    public Pilha() {
        this.elementos = new String[CAPACIDADE];
        this.tamanho = 0;
    }

    public void empilha(String elemento) {
        if (tamanho == CAPACIDADE) {
            throw new RuntimeException("Pilha cheia! (overflow)");
        }
        elementos[tamanho] = elemento;
        tamanho++;
    }

    public String desempilha() {
        if (estaVazia()) {
            throw new RuntimeException("Pilha vazia! (underflow)");
        }
        tamanho--;
        return elementos[tamanho];
    }

    public String topo() {
        if (estaVazia()) {
            throw new RuntimeException("Pilha vazia!");
        }
        return elementos[tamanho - 1];
    }

    public boolean estaVazia() {
        return tamanho == 0;
    }

    public int tamanho() {
        return tamanho;
    }
}
```

```

@Override
public String toString() {
    StringBuilder sb = new StringBuilder("[");
    for (int i = 0; i < tamanho; i++) {
        sb.append(elementos[i]);
        if (i < tamanho - 1) sb.append(", ");
    }
    sb.append("]");
    return sb.toString();
}
}

```

Exercício 2: Testando a Pilha

```

public class TestePilha {
    public static void main(String[] args) {
        Pilha pilha = new Pilha();

        pilha.empilha("A");
        pilha.empilha("B");
        pilha.empilha("C");

        System.out.println("Topo: " + pilha.topo());

        String removido = pilha.desempilha();
        System.out.println("Removido: " + removido);

        System.out.println("Topo: " + pilha.topo());
    }
}

```

Saída esperada:

```

Topo: C
Removido: C
Topo: B

```

Exercício 3: toString()

Já implementado na classe `Pilha` acima com `@Override`.

Exemplo de saída:

```
[A, B, C]
```

Parte 2 — Exercícios de Aplicação

Exercício 4: Inverter uma Palavra

```
public class InverterPalavra {
    public static void main(String[] args) {
        String palavra = "hello";
        Pilha pilha = new Pilha();

        for (int i = 0; i < palavra.length(); i++) {
            pilha.empilha(String.valueOf(palavra.charAt(i)));
        }

        StringBuilder resultado = new StringBuilder();
        while (!pilha.estaVazia()) {
            resultado.append(pilha.desempilha());
        }

        System.out.println(resultado.toString());
    }
}
```

Saída esperada:

```
olleh
```

Exercício 5: Verificar Palíndromo

```
public class Palindromo {
    public static void main(String[] args) {
        verificar("arara");
        verificar("casa");
    }

    public static void verificar(String palavra) {
        Pilha pilha = new Pilha();

        for (int i = 0; i < palavra.length(); i++) {
            pilha.empilha(String.valueOf(palavra.charAt(i)));
        }

        StringBuilder invertida = new StringBuilder();
        while (!pilha.estaVazia()) {
            invertida.append(pilha.desempilha());
        }

        if (palavra.equals(invertida.toString())) {
```

```

        System.out.println(palavra + " → É palíndromo");
    } else {
        System.out.println(palavra + " → Não é palíndromo");
    }
}
}

```

Saída esperada:

```

arara → É palíndromo
casa → Não é palíndromo

```

Exercício 6: Inverter Ordem das Palavras

```

public class InverterFrase {
    public static void main(String[] args) {
        String frase = "eu gosto de java";
        String[] palavras = frase.split(" ");
        Pilha pilha = new Pilha();

        for (String palavra : palavras) {
            pilha.empilha(palavra);
        }

        StringBuilder resultado = new StringBuilder();
        while (!pilha.estaVazia()) {
            resultado.append(pilha.desempilha());
            if (!pilha.estaVazia()) resultado.append(" ");
        }

        System.out.println(resultado.toString());
    }
}

```

Saída esperada:

```

java de gosto eu

```

Parte 3 — Questões Objetivas

#	Pergunta resumida	Resposta
7	Princípio LIFO	b) Último que entra é o primeiro que sai

#	Pergunta resumida	Resposta
8	push(A,B,C), pop(), push(D)	c) A, B, D
9	[A,B,C,D], pop(), pop(), push(E)	d) E
10	push(1,2,3,4), quatro pop()	b) 4, 3, 2, 1
11	push() em pilha cheia	b) overflow
12	[A,B], três pop()	c) underflow
13	Operação que insere no topo	c) push
14	Operações fundamentais	a) push e pop
15	Índice do topo em vetor	c) tamanho - 1

Justificativas

Q7: LIFO = *Last In, First Out*. O último elemento empilhado é o primeiro a ser removido.

Q8: Estado passo a passo: push(A) → [A] | push(B) → [A,B] | push(C) → [A,B,C] | pop() remove C → [A,B] | push(D) → [A,B,D].

Q9: [A,B,C,D] com D no topo. pop() remove D → [A,B,C]. pop() remove C → [A,B]. push(E) → [A,B,E]. Topo = **E**.

Q10: Pilha empilha 1,2,3,4. O topo é 4. Cada pop() retira do topo, então a ordem de remoção é 4,3,2,1.

Q11: Tentar inserir em pilha que já está na capacidade máxima causa **overflow**.

Q12: [A,B] tem apenas 2 elementos. Três pop() tentam remover mais do que existe, causando **underflow**.

Q13: push insere no topo. enqueue / dequeue são operações de fila (Queue).

Q14: As duas operações essenciais de uma pilha são push (empilhar) e pop (desempilhar).

Q15: Em vetores com índice base 0, se há tamanho elementos, o último (topo) está em tamanho - 1.